

A Conditional Source-to-Abstract Functional-Correctness Theorem for FreeRTOS V6.1.1 vTaskDelay(2)

Mathematical progress report - blind Isabelle/HOL reconstruction Status date: 1 August 2026 **Result label:** conditional real-source functional-correctness theorem: green; concrete frozen-build-layout P2 non-vacuity witness: open

Draft provenance. The initial narrative draft was prepared with ChatGPT Pro; all claims in this report were audited against the local Isabelle theories and run ledgers. ChatGPT Pro supplied design review and prose only and is outside the trusted proof chain.¹

Abstract

This report describes a mechanically checked refinement result for one non-trivial execution path of the FreeRTOS V6.1.1 scheduler: a running task at priority 2 calls `vTaskDelay(2)` when the tick count is 5. The path removes the running task's generic list item from the priority-2 ready list, writes wake time 7, inserts the item into the current delayed list, resumes the scheduler through its quiet branch, and issues one proof-port yield. The abstract poststate is deliberately phase-indexed: it is a `YieldPending` endpoint, not a settled post-switch scheduler state. Thus the ready-priority cache and current-task pointer may still identify the pre-yield task even though that task has already migrated to the delayed list.

The development separates three proof obligations. First, a source-level theorem symbolically executes the generated semantics of the real `vTaskDelay` body once and identifies an exact final global state. Second, a pure byte-heap postrelation proves the representation of all eight physical scheduler list roots after the remove-key-write-insert transformation. Third, an assembly theorem combines these results with decoder, delayed-list-role, scalar, current-task, and boundary relations to refine the abstract operation. All three layers have exit-zero Isabelle builds with `quick_and_dirty=false`; their theory files are fixed by SHA-256 digests recorded below.

The result is conditional. Its hypotheses contain a scheduler endpoint relation and a detailed source footprint. The footprint constrains guards, containment, and separation, but the generated addressed-data constants are not yet instantiated from a particular linked FreeRTOS binary. Consequently the proof establishes functional correctness for every concrete state satisfying those hypotheses, but does not yet prove that one frozen ELF image supplies such a state. Constructing an artifact-bound data-layout certificate is the next required milestone. Reachability of the certified P2 state from boot and scheduler initialisation is a later, separate milestone.

1. Scope, statement discipline, and trusted chain

The target is neither whole-kernel correctness nor a claim about every argument of `vTaskDelay`. It is the positive-delay P2 path with the fixed 32-bit argument 2 and the fixed abstract witness described in Section 3. The proof opens the generated source body for this path and composes previously checked exact contracts for suspension, ready-list removal, the generated wake-key write, ordered insertion into an empty delayed list, quiet resume, and the proof-port yield.

The semantic judgment used below is written

$$\begin{aligned} & \text{RunsTo}(f, c, Q) \\ & \equiv \\ & \text{succeeds}(f, c) \\ & \wedge \\ & \forall r \ t. \text{reaches}(f, c, r, t) \Rightarrow Q(r, t). \end{aligned}$$

It is important that this judgment contains both positive execution and universal postcondition components. A correspondence theorem or a universal claim over an empty set of executions would not be sufficient. Here, `succeeds` rules out that vacuity, while the `reaches` implication constrains every result and final state admitted by the generated monadic semantics.

The checked chain for the current result is:

```
frozen FreeRTOS V6.1.1 source slice
  |
  v
CParser / AutoCorres2 translation
  |
  v
generated monadic semantics of the real vTaskDelay body
  |
  v
Isabelle/HOL proof scripts
  |
  v
Isabelle kernel-checked theorem objects
```

The C compiler and harness traces are useful for invariant discovery and regression detection, but they are not proof rules. ChatGPT Pro is likewise not a proof rule. The authoritative evidence for the claims in this report is the checked theory graph, the exit status and configuration of each bounded Isabelle build, the no-forbidden-construct scan, and the

recorded hashes. A future ELF symbol extractor may be untrusted if it emits candidate data that are independently checked; its output must not enter the logic as an axiom.

2. Mathematical objects and notation

The abstract scheduler state is a record containing live tasks, priorities, optional wake times, ready and delayed rings, delayed-list roles, pending and suspended rings, the 32-bit tick, suspension and missed-tick counters, a missed-yield flag, the top-ready cache, the current task, the overflow count, and the proof-port yield count. A node is either a task's generic list item or event list item. Only generic nodes occur in the P2 ready and delayed rings.

Symbol	Meaning
I	abstract task identifier P2_IDLE
U	abstract task identifier P2_RUN
$G(t)$	generic list node belonging to task t
ε	empty abstract node ring, with no cursor
$[n]_{\rightarrow n}$	singleton ring containing n , with cursor at n
$[n]_{(\odot)^k}$	singleton ordered ring containing n , no cursor, item key k
a_0, a_1	the abstract P2 prestate and poststate
D	concrete-pointer decoder and encoder package for the two P2 tasks
R	scheduler root package: four ready roots plus four non-ready roots
c, t	generated scheduler global states
$h(c)$	byte heap <code>hrs_mem(t_hrs'(c))</code> extracted from c
$E_{\varphi}(D, R, c, a)$	endpoint relation at phase φ
$F(D, R, H)$	P2 source-footprint predicate on heap H
$L(D, H, l, q)$	physical list at pointer l represents abstract ring q under decoder D
$Result()$	normal unit-valued return of the generated monad
$+_{(32)}$	addition modulo $2^{(32)}$

The endpoint relation packages seven concrete-to-abstract components: abstract core well-formedness, decoder correctness, all-list representation, delayed-root roles, scalar fields, the current-task pointer, and the scheduler/port boundary. At phase `StableRunning`, it additionally requires the stronger settled invariant. At phase `YieldPending`, it requires core well-formedness, suspension depth and missed ticks equal to zero, an empty pending ring, and a positive yield count. This phase distinction is load-bearing; it prevents the proof from asserting a scheduler invariant at an intermediate boundary where the source has requested but not yet performed a context switch.

3. The P2 abstract transition

The P2 witness contains exactly two live tasks. Their priorities are

$$\begin{aligned}\pi(I) &= 0, \\ \pi(U) &= 2.\end{aligned}$$

The initial tick is 5 and the delay is 2, so the 32-bit wake time is

$$w = 5 +_{(32)} 2 = 7.$$

Because $7 \nless 5$, this is the non-wrapping branch: the node is inserted into delayed list A, which is the current delayed-list role in the prestate. The abstract operation removes $G(U)$ from ready priority 2, records wake time 7, performs ordered insertion into the current delayed ring, and increments the yield count. Its calculation is

$$task_delay_abs(2, a_0) = a_1.$$

The complete observable state comparison is as follows. “Unchanged” means equality of the corresponding total field, not merely equality on the two named tasks.

Component	a_0 (p2_pre)	a_1 (p2_post)
Live tasks	I, U	I, U
Priority map	$I \mapsto 0, U \mapsto 2$	unchanged
Wake map	every task $\mapsto None$	$U \mapsto Some(7)$; all others $\mapsto None$
Event-waiting set	$\emptyset$	$\emptyset$
Ready root 0	$[G(I)]_{\rightarrow G(I)}$	unchanged
Ready root 1	ε	ε
Ready root 2	$[G(U)]_{\rightarrow G(U)}$	ε
Ready root 3	ε	ε
Delayed A	ε	$[G(U)]_{(\odot)^{(7)}}$
Delayed B	ε	ε
Current delayed role	A	A
Pending-ready ring	ε	ε

Component	a_0 (p2_pre)	a_1 (p2_post)
Suspended ring	ϵ	ϵ
Tick	5	5
Missed ticks	0	0
Suspension depth	0	0
Missed-yield flag	false	false
Top-ready cache	2	2
Current task	$Some(U)$	$Some(U)$
Overflow count	0	0
Yield count	0	1

The poststate satisfies the scheduler's core invariant but not its settled invariant. That is expected. The priority-2 ready ring is already empty, while the top-ready cache and current-task field have not yet been recomputed by `vTaskSwitchContext`. The mathematical endpoint is therefore `YieldPending`; the report makes no claim that the context switch has already occurred.

4. Representation layers

The source semantics operates over generated C structures and a byte-addressed heap, whereas the abstract model uses finite node rings and task identifiers. The proof connects them through explicit layers rather than treating C pointers as abstract nodes.

The decoder package D contains a task-to-TCB pointer function, a partial inverse from TCB pointers to task identifiers, and a partial decoder from raw list-item pointers to generic or event nodes. On the live set it is injective and round-trips both TCB pointers and the two embedded list-item pointers. Conversely, a successful decode identifies the unique corresponding live task and embedded item.

The root package is the ordered vector

$$R = [R_0, R_1, R_2, R_3, D_A, D_B, P, S_{(root)}],$$

where R_i is ready priority i , D_A and D_B are the two physical delayed roots, P is the pending-ready root, and $S_{(root)}$ is the suspended root. The generated role pointers select D_A as current and D_B as overflow in the P2 witness.

For each root, the relation L first establishes a raw cyclic-list representation in the heap and then relabels raw item pointers through D to obtain the abstract ring. This separation is needed because the source-derived list functions and the scheduler translation inhabit distinct generated structure namespaces even though their relevant layouts agree. Audited ABI read and write bridges connect their field addresses and heap observations; the scheduler theorem does not silently identify the two generated types.

The scalar relation maps 32-bit source values to the corresponding abstract fields, with `unat` conversions where the abstract model uses natural numbers. It also maps the concrete missed-yield word to a Boolean and the proof-port yield counter to the abstract yield count. The current-task relation decodes `pxCurrentTCB`; the boundary relation fixes a running scheduler, critical depth zero, and interrupts enabled at the theorem boundary.

5. The source footprint and why it is still conditional

The footprint $F(D, R, H)$ is a semantic memory contract, not a claim that the generated constants already denote the addresses in a linked binary. It requires:

1. R equals the generated scheduler-root package.
2. The eight roots in R are distinct, individually guarded, and pairwise region-disjoint.
3. The two TCB pointers for I and U are distinct, guarded, and have disjoint 68-byte TCB regions.
4. Each TCB region is disjoint from every root region.
5. In each TCB, the generic and event list items are guarded, contained within that TCB region, and disjoint from each other.
6. The generated ABI places the 20-byte generic item at TCB offset 4 and the 20-byte event item at offset 24; these intervals lie inside the 68-byte TCB object.
7. Both delayed-list sentinels have the maximum key required by the ordered insertion branch.

In compact form, with Reg_L and Reg_T denoting list-root and TCB byte intervals, the central separation obligations include

$$\begin{aligned}
& \text{distinct}(R), \\
& \forall x \in R. \text{guard}(x), \\
& \forall x \neq y \in R. \\
& \text{Reg_L}(x) \cap \text{Reg_L}(y) = \emptyset, \\
& \text{tp_l} \neq \text{tp_u}, \\
& \text{guard}(\text{tp_l}) \wedge \text{guard}(\text{tp_u}), \\
& \text{Reg_T}(\text{tp_l}) \cap \text{Reg_T}(\text{tp_u}) = \emptyset, \\
& \forall \text{tp} \in \text{tp_l}, \text{tp_u}, x \in R. \\
& \text{Reg_T}(\text{tp}) \cap \text{Reg_L}(x) = \emptyset.
\end{aligned}$$

These assumptions are strong enough for the local alias, guard, and frame proofs. They are not yet an existence theorem. In the present translation, addressed data globals are unconstrained HOL constants, and the existing global-address locale constrains function addresses rather than all scheduler data objects. The missing result must bind the root and TCB addresses to one exact tuple of source, configuration, compiler, assembler, linker flags, linker script, ELF, and supporting symbol/layout data. A synthetic assignment would prove logical satisfiability only; it would not prove the layout of the frozen artifact.

6. Exact source execution and heap transformers

Let the initial generated global state be c , and define the initial byte heap and the relevant concrete pointers by

$$\begin{aligned}
H_0 &\equiv h(c) \\
&= \text{hrs_mem}(t_hrs'(c)), \\
tp &\equiv \text{sd_tcb_ptr}(D, U), \\
p &\equiv \text{abi_generic_list_item_ptr}(tp), \\
l_A &\equiv \text{abi_list_ptr}(sr_delayed_a(R)).
\end{aligned}$$

The exact heap sequence checked by the source proof is

$$\begin{aligned}
H_r & \\
&\equiv \text{raw_remove_concrete_heap}(H_0, p), \\
H_k & \\
&\equiv \text{p2_remove_then_wake_heap}(H_0, D) \\
&= \text{scheduler_generic_item_key_heap} \\
&\quad (H_r, tp, 5 + (32)2) \\
&= \text{scheduler_generic_item_key_heap}(H_r, tp, 7), \\
H_f & \\
&\equiv \text{p2_remove_wake_insert_heap}(H_0, D, R) \\
&= \text{raw_ordered_insert_empty_heap}(H_k, l_A, p).
\end{aligned}$$

Thus H_r is the heap immediately after singleton removal from ready root 2, H_k additionally contains the generated write of wake key 7, and H_f additionally contains the empty-to-singleton ordered insertion into delayed A. These are total heap transformers. Their supporting locality facts show that writes to the active ready root, the running task's generic item, and delayed A preserve the other roots and the idle task's generic item as required by the final relation.

The final generated global state is exactly

$$\begin{aligned}
& S(c, D, R) \\
& = \\
& \text{Yield}(\\
& \quad \text{ResumeQuiet}(\\
& \quad \quad \text{Mem}(\\
& \quad \quad \quad H_f, \\
& \quad \quad \quad \text{Suspend}(c) \\
& \quad \quad) \\
& \quad) \\
&).
\end{aligned}$$

Here `Suspend` increments the suspension word from 0 to 1, `Mem` replaces only the byte-heap component with H_f , `ResumeQuiet` decrements the suspension word from 1 to 0 without processing pending work, and `Yield` increments the proof-port yield counter from 0 to 1. Written with the definitions used in the theory, this is

$$\begin{aligned}
& \text{p2_yield_state} \\
& (\text{p2_resume_quiet_state} \\
& \quad (\text{scheduler_mem_state} \\
& \quad \quad (H_f, \text{p2_suspend_state}(c))))).
\end{aligned}$$

Theorem Box 1 - exact source state (`scheduler_vTaskDelay_2_p2_exact_state`). For every D, R, c , if

$$E_(\text{StableRunning})(D, R, c, a_0)$$

$$\wedge F(D, R, H_0),$$

then

$$\text{RunsTo}(\text{vTaskDelay}'(2), c,$$

$$\lambda(r, t). r = \text{Result}() \wedge t = S(c, D, R)$$

$$).$$

The proof unfolds the real generated `vTaskDelay` body once. It does not replace the caller with a hand-written model. Exact contracts for the called operations are then composed at the actual intermediate states, including the generated nested field write. This arrangement avoids a second large VCG while retaining a precise final state.

7. The all-eight-roots postrelation

The heap transformer alone is not an abstract correctness result. The next layer proves that each physical root in H_f represents its intended component of a_1 . Using `dec_D` for the node decoder and suppressing the routine ABI pointer coercions, the postrelation is the conjunction

$$L(D, H_f, R_0, [G(I)]_(\leadsto G(I)))$$

$$\wedge L(D, H_f, R_1, \epsilon)$$

$$\wedge L(D, H_f, R_2, \epsilon)$$

$$\wedge L(D, H_f, R_3, \epsilon)$$

$$\wedge L(D, H_f, D_A, [G(U)]_(\emptyset) \wedge (7))$$

$$\wedge L(D, H_f, D_B, \epsilon)$$

$$\wedge L(D, H_f, P, \epsilon)$$

$$\wedge L(D, H_f, S_(\text{root}), \epsilon).$$

The ready-0 case is not reducible to a root-only frame: its singleton relation observes both the list root and the embedded generic item of I . The proof therefore establishes preservation of that entire item as well. Ready 2 uses the exact post-removal empty representation and frames it across the later key write and delayed insertion. Delayed A uses the ordered empty-insertion relation with key 7. The remaining five empty roots are carried across by pairwise region separation.

The checked pure relation result is named `p2_remove_wake_insert_lists_rel`. In mathematical form, if decoder and pre-list relations hold, the footprint holds, and a generated state c' has heap H_f , then

$$\text{DecodeRel}(D, a_0)$$

$$\wedge \text{ListsRel}(D, R, c, a_0)$$

$$\wedge F(D, R, H_0)$$

$$\wedge h(c') = H_f$$

$$\Rightarrow$$

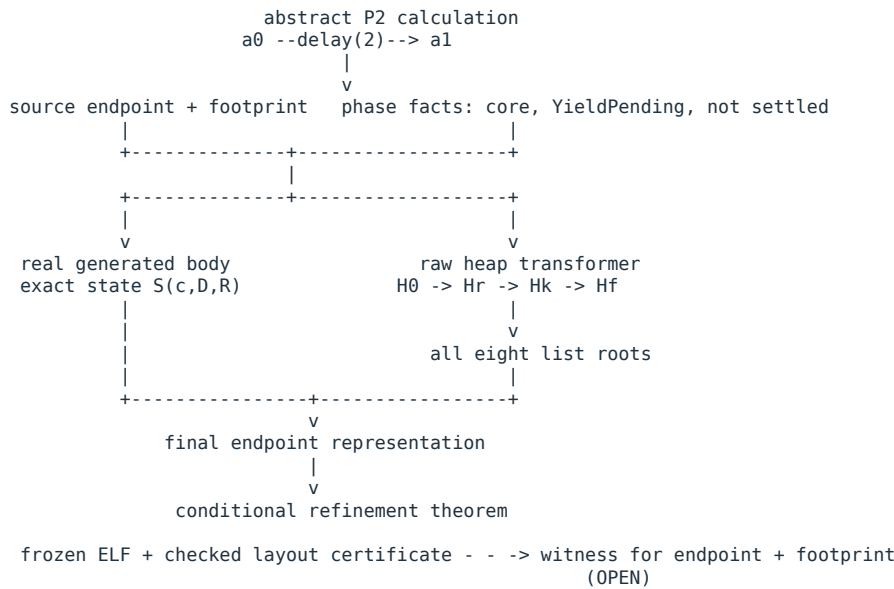
$$\text{ListsRel}(D, R, c', a_1).$$

This layer is deliberately pure Isabelle relation assembly. It does not execute the C body again. Separating operational execution from postrelation assembly makes the proof graph auditable and keeps the expensive generated-body reasoning confined to one source certificate.

8. Final source-to-abstract refinement

The last layer combines the exact source state with the eight-root postrelation and direct proofs for the non-list components. Decoder correctness is unchanged because the live-task set and pointer interpretation are unchanged. Delayed-root roles remain A-current/B-overflow. The scalar relation observes tick 5, suspension depth 0, no missed ticks or missed yield, top-ready cache 2, two live tasks, overflow count 0, and yield count 1. The current pointer remains the TCB of U . The boundary again has a running scheduler, critical depth zero, and interrupts enabled.

The proof dependency is summarized below. Solid arrows denote already checked implications. The dashed future edge is exactly the open non-vacuity task and is not used to label the conditional theorem green.



Theorem Box 2 - conditional functional correctness (`scheduler_vTaskDelay_2_p2_refines_task_delay_abs`). For every D, R, c , if

$$E_(\text{StableRunning})(D, R, c, a_0) \\ \wedge F(D, R, h(c)),$$

then

$$\text{RunsTo}(\text{vTaskDelay}'(2), c, \lambda(r, t). \\ r = \text{Result}() \\ \wedge \\ E_(\text{YieldPending})(D, R, t, \text{task_delay_abs}(2, a_0)) \\).$$

Since $\text{task_delay_abs}(2, a_0) = a_1$, every admitted source execution returns normally in a concrete state representing a_1 at the `YieldPending` boundary.

A checked intermediate result also establishes

$$E_(\text{YieldPending})(D, R, S(c, D, R), a_1) \\ \wedge \neg \text{settled_wf}(a_1).$$

The negated settled predicate is a positive description of the phase boundary, not a proof failure. The later context-switch caller is responsible for selecting the next ready task and restoring the settled scheduler condition.

9. Reproducibility and checker evidence

The four principal P2 runs are listed below. Each was a bounded Isabelle build with exit code 0 and `quick_and_dirty=false`. The digest shown is the SHA-256 of the corresponding theory source at the recorded green run.

Proof layer	Run identifier	Final wall time	Result	Theory SHA-256
Pure abstract P2 transition and phase witness	20260731Tscheduler-p2-06-post-witness	12.597 s	exit 0; QAD false	FB03C1FDE6BEC66357F4E7185838423B8867D8999E2CD1422CF2064DC0C48236
Exact real-source P2 state	20260801Tscheduler-p2-delay-source-09-canonical-states	35.056 s	exit 0; QAD false	7BF8623FE099104EA4276D0119CCD6C7B23A0F94ABF04B87FAAF3AEB1DD6C0
All-eight-roots postrelation	20260801Tscheduler-p2-post-relation-09-nat-index	38.618 s	exit 0; QAD false	CF5CCB728150BC4B185CEAE448E9E2FAB0C9C88E1CCBCDA1F79E2A67AD7375B8
Final conditional refinement assembly	20260801Tscheduler-p2-delay-refinement-04-final-heap	39.603 s	exit 0; QAD false	43BC41254C609DF8302F38113F662912E2FD99C72D0A612E5A72925E5AA1C9E7
Current portable dependency-closure replay	20260801Tpublish-portable-refinement-02	385.146 s	exit 0; QAD false	final theory unchanged: 43BC41254C609DF8302F38113F662912E2FD99C72D0A612E5A72925E5AA1C9E7

The exact-source development took nine checker calls and 413.312 seconds in total, of which the last call was green. The final list-relation development took nine calls and 367.365 seconds, and the final assembly took four calls and 169.437 seconds. Those totals measure proof discovery as well as final replay; the final-wall column is the more useful reproducibility expectation for the fixed successful theory.

The current-tree replay was run after replacing four machine-specific CParser wrapper paths by theory-master-directory resolution. Its status SHA-256 is 6D45F563CB84212812C59F5A723DD17177E03FF36328D6FFAF96AAA3FAB78B5E, and its stdout SHA-256 is 594A6C89502091A013CB9A6D71EDC6DB1AB76A5C174D869F15DB2687F4B95C89. This closes the evidence-timing gap between the portable parser snapshot and the final refinement theorem.

The source-to-ITP mapping validator passes, 65 repository unit tests pass, and the theory/proof-port/build-script forbidden-pattern scan is clean. The mapping manifest validates the evidence inventory currently registered in its schema; it still records the preceding count of eleven source-to-abstract refinements and has not yet promoted this new P2 positive-delay theorem as a twelfth rung. The theorem and run evidence above are therefore reported directly, not presented as a manifest-registered rung. In particular, the accepted proof does not depend on `sorry`, `oops`, `admit`, an axiom declaration, skip-proof mode, an oracle shortcut, or `quick_and_dirty=true`. Hashes are provenance aids rather than logical premises: they make it possible to detect whether a later report is still describing the checked files.

10. Claim taxonomy, open obligations, and tooling

The following taxonomy prevents useful evidence from being promoted beyond what it establishes.

Class	Current content	Status
Kernel-checked pure mathematics	abstract two-tick P2 transition; core and phase facts	green
Kernel-checked source execution	positive execution of generated <code>vTaskDelay</code> (2) with one exact final state	green under endpoint and footprint premises
Kernel-checked representation	all eight roots plus decoder, roles, scalars, current task, and boundary	green under the same premises
Kernel-checked refinement	real source result represents the abstract delay result at <code>YieldPending</code>	conditional theorem green
Executable evidence	frozen-source harness traces and regression checks used for invariant discovery	useful, but not a theorem
Artifact-bound satisfiability	concrete addresses and byte heap from one frozen linked FreeRTOS build satisfy the premises	open
Initialisation reachability	the artifact-bound P2 state is reachable from boot and scheduler setup	later milestone
Scheduler-wide correctness	arbitrary tasks, priorities, delays, wrap branches, context switches, and all callers	out of scope for this report

10.1 The next proof obligation

The next required result is an artifact-bound existential witness of the form

$$\begin{aligned} &\exists D, c. \\ &E_(\text{StableRunning}) \\ &(D, R_(\text{generated}), c, a_0) \\ &\wedge \\ &F(D, R_(\text{generated}), h(c)), \end{aligned}$$

where every generated root constant is connected, by checked certificate facts, to symbols in one frozen ELF. The certificate must record exact source and configuration hashes, compiler/assembler/linker versions and flags, linker script, ELF, and supporting map/DWARF/symbol-table data. It must check symbol identity, object extents, alignment, containment, pairwise interval separation, and the embedded list-item offsets.

A practical architecture is: an untrusted extractor reads the ELF and emits candidate addresses and sizes; a small independently reviewable checker validates the symbol and interval ledger; Isabelle checks the resulting concrete arithmetic and interprets a data-layout locale; finally, a concrete two-task decoder, heap, and globals state instantiate the existential statement. If no verified ELF parser is used, the residual extractor and binary-analysis trust boundary must be stated explicitly.

10.2 Do additional symbolic execution, VCG, or SMT tools help?

The present source theorem already uses the CParser/AutoCorres2-generated monadic semantics together with a verification-condition generator. No missing operational path remains for the P2 result, so adding a second symbolic executor or a generic SMT backend would duplicate work rather than close the outstanding premise. The difficult completed obligations were typed ABI bridging, byte-heap locality, alias separation, and assembly of the physical-to-abstract relation; they required project-specific invariants and frame lemmas rather than generic first-order search.

An additional tool can nevertheless accelerate the next milestone if it is aimed at the actual gap: a deterministic ELF/link-map/DWARF extractor plus a small certificate checker. SMT may be used as an untrusted convenience for proposing interval arithmetic, but the accepted address, extent, and separation facts should be replayed by Isabelle or another independently reviewable checker. The artifact identity and the connection between generated constants and ELF symbols are more important than solver power.

Boot reachability should not be folded into the layout certificate. Layout answers *where the objects are* and supplies one state satisfying the memory premises. Reachability answers *whether the official initialisation path can construct that state*. Keeping the two milestones separate preserves an auditable statement of what has and has not been proved.

Conclusion

The P2 development has crossed the main functional-correctness boundary. The real generated `vTaskDelay'(2)` semantics has a positive exact execution theorem; the corresponding concrete heap is shown to represent the intended poststate at all eight scheduler list roots; and the assembled final result refines the abstract two-tick delay transition. The phase is correctly recorded as `YieldPending`, with one yield request and no claim that context switching has already settled the scheduler.

The remaining blocker is narrower and qualitatively different from symbolic execution. It is the absence of a checked, frozen-build data-layout instantiation for the addressed scheduler globals and TCBs. Until that existential witness is constructed, the strongest accurate summary remains: **conditional real-source functional-correctness theorem green; frozen-build-layout P2 non-vacuity open; boot reachability later.**

Appendix A. Compact proof obligations

For reference, the completed logical chain can be compressed to the following five formulas:

$$task_delay_abs(2,a_0)=a_1.$$

$$E_StableRunning(D,R,c,a_0) \wedge F(D,R,H_0)$$

$\Rightarrow$

$$RunsTo(vTaskDelay'(2),c, \\ \lambda(r,t). r=Result() \wedge t=S(c,D,R)).$$

$$DecodeRel(D,a_0) \wedge ListsRel(D,R,c,a_0)$$

$$\wedge F(D,R,H_0) \wedge h(c')=H_f$$

$$\Rightarrow ListsRel(D,R,c',a_1).$$

$$E_StableRunning(D,R,c,a_0) \wedge F(D,R,H_0)$$

$\Rightarrow$

$$E_YieldPending(D,R,S(c,D,R),a_1) \\ \wedge \neg settled_wf(a_1).$$

$$E_StableRunning(D,R,c,a_0) \wedge F(D,R,H_0)$$

$\Rightarrow$

$$RunsTo(vTaskDelay'(2),c, \\ \lambda(r,t). r=Result() \wedge \\ E_YieldPending(D,R,t, \\ task_delay_abs(2,a_0))).$$

Appendix B. Interpretation checklist

- “Real source” means that the proof opens the generated semantics of the frozen C function body; it does not mean that the linked deployment image has already been instantiated.
 - “Exact state” means every admitted final generated state equals the displayed transformer result, together with positive success.
 - “All eight roots” means ready priorities 0-3, delayed A, delayed B, pending-ready, and suspended.
 - “Refinement” means preservation of the stated endpoint relation to the abstract operation, at the explicitly named phase.
 - “Conditional” refers precisely to the endpoint and source-footprint premises whose frozen-build preimage remains to be constructed.
 - “Non-vacuity open” refers to the artifact-bound existence of those premises, not to the source run under them; the latter is already positive.
 - “Boot reachability later” means no present claim connects the P2 prestate to the complete official initialisation and scheduling history.
1. No external generated proof, attachment, source artifact, or theorem from ChatGPT Pro was admitted. Its contribution was an initial prose/design pass; the report's formulas, theorem names, run data, and limitations were checked against the local repository.